Neural Networks for Regression and Classification
FYS-STK3155 - Project 2

Carl Petter Duedahl, Ole Morten Gården, and Ingi Benedikt Jónasson
University of Oslo
(Dated: 9th November 2026)

While linear regression methods rely on features chosen by hand, neural networks can learn their own features from the raw data. In this project we implemented feed-forward neural networks with backpropagation from scratch, and tried out sigmoid, ReLU and Leaky ReLU as activation functions, L_1 and L_2 penalties and four gradient-based optimisers. We first applied our network to the one-dimensional Runge function with added Gaussian noise (standard deviation 0.1), to compare it with the OLS, Ridge and Lasso models from a previous project. Our best network gave a test MSE of 0.011, on par with Ridge and Lasso (0.013) and OLS (0.015). We found that the optimiser and the learning rate mattered more than the architecture. Deep sigmoid networks trained with plain SGD, and plain gradient descent, often failed to learn, while Adam and RMSprop worked reliably with small learning rates. We then classified handwritten digits from the MNIST dataset. Here our network with one hidden layer of size 200 reached a test accuracy of 98.0 %, compared with 92.6 % for logistic regression, in agreement with Scikit-Learn. Our results suggest that neural networks are most useful when good features are not known in advance.

I. INTRODUCTION

In recent years, neural networks have gotten a rise in attention in the machine learning scene, partly because of the rise in computing power [1]. They are flexible, as they can learn their features from raw input. This is different from the linear regression models from our previous project, Project 1 [2], where we had to choose these features ourselves, e.g. as powers of x in a polynomial fit.

In this project we used the feed-forward neural network method with backpropagation, explained in section II C, to tackle one regression problem and one classification problem. The regression problem was to make a fit of the Runge function, which is known to be hard to approximate with polynomials (section II A). We tuned different parts of the neural network, like the learning rate γ, the optimiser, the number of hidden layers and their size, the L_1 and L_2 penalties and whether to use the sigmoid, ReLU or Leaky ReLU as activation function. First we did this by hand, then we used cross-validation. While developing our neural network, we also compared with our results from Project 1 [2], where we used OLS, Ridge and Lasso to fit the same function. Our tuning process can be seen in Sections III B–III E.

In our second problem we used the MNIST data set [3], which consists of 70 000 pictures of handwritten Arabic numerals, and our model should classify these pictures as the correct digits. Here we used logistic regression, i.e. our network without hidden layers, as a baseline, and compared both models with Scikit-Learn [4].

The structure of our report is a Theory and Methods section in section II, which introduces some core concepts like the cost functions, the network, the backpropagation algorithm and the optimisers, and section II F describes how we implemented our methods. In section III we first verify the code, and then present and discuss our results and compare them to Project 1 [2]. Finally, section IV summarises our findings and discusses how we could further improve these methods in the future. All code, data and figures are available on our GitHub repository: https://github.com/carlpd/FYS-STK, in the folder `Project2`.

II. THEORY AND METHODS

A. The Runge function

We use the Runge function

$$f(x) = \frac{1}{1+25x^2}, \qquad x \in [-1, 1]. \tag{1}$$

Although it is smooth and bounded on the real line, it has poles at $x = \pm i/5$ in the complex plane, so its Maclaurin series converges only for $|x| < 1/5$. Polynomials therefore approximate it poorly near the ends of the interval, where high-degree polynomials tend to oscillate (Runge's phenomenon). This makes it a demanding test case for polynomial regression.

B. Cost functions and their derivatives

To judge the quality of a fit, we need a measure of how far the predictions are from the data. This is the role of the *cost function* (or *loss function*) $C(\mathbf{y}, \tilde{\mathbf{y}})$, which takes the observed data $\mathbf{y} \in \mathbb{R}^n$ and the predictions $\tilde{\mathbf{y}} \in \mathbb{R}^n$ for n data points, and returns a single number measuring the "distance" between them [5].

Since the predictions $\tilde{\mathbf{y}}$ depend on the parameters $\boldsymbol{\theta}$ of the model, we also write the cost as $C(\boldsymbol{\theta})$, and training the model means minimising it with respect to $\boldsymbol{\theta}$. The gradient of C tells us in which direction to change the parameters to lower the cost, and it is the basis of all the gradient methods in section II D. For each cost function below, we therefore also give its gradient.

1. Mean-squared error

For regression, the natural choice is the *mean-squared error* (MSE),

$$\mathrm{MSE}(\mathbf{y}, \tilde{\mathbf{y}}) = \frac{1}{n}\sum_{i=0}^{n-1}(y_i - \tilde{y}_i)^2 = \frac{1}{n}\|\mathbf{y} - \tilde{\mathbf{y}}\|_2^2. \quad (2)$$

Its gradient with respect to the predictions,

$$\nabla_{\tilde{\mathbf{y}}}\mathrm{MSE}(\mathbf{y}, \tilde{\mathbf{y}}) = \frac{2}{n}(\tilde{\mathbf{y}} - \mathbf{y}), \quad (3)$$

is the starting point of backpropagation (section II C 4). When training the network, we follow the lecture notes [6] and minimise half the MSE, $C = \mathrm{MSE}/2$. The factor 2 then cancels, and the gradient with respect to the output is simply $(\tilde{\mathbf{y}} - \mathbf{y})/n$. The minimum is unchanged, and apart from the training cost in Figure 3, all MSE values we report are the full MSE.

2. Cross entropy

In binary classification the targets are $y_i \in \{0, 1\}$, and a logistic model with inputs $\mathbf{x}_i$ (the rows of the design matrix X) and weights $\boldsymbol{\theta}$ predicts the probability $p_i = \sigma(\mathbf{x}_i^T\boldsymbol{\theta})$ that $y_i = 1$. The corresponding cost is the *cross entropy*,

$$C(\boldsymbol{\theta}) = -\frac{1}{n}\sum_{i=0}^{n-1}\left[y_i \log p_i + (1 - y_i)\log(1 - p_i)\right]. \quad (4)$$

This is the *negative log-likelihood* of the data when each target is a Bernoulli variable with probability p_i. For each sample only one of the two terms is non-zero. A confident and correct prediction gives a cost close to zero, while a confident mistake, such as $p_i \to 0$ when $y_i = 1$, gives a large cost. The gradient is

$$\nabla_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) = -\frac{1}{n}X^T\left(\mathbf{y} - \sigma(X\boldsymbol{\theta})\right), \quad (5)$$

where the *sigmoid function* $\sigma(z) = 1/(1 + e^{-z})$ acts element by element. Like the MSE, the cross entropy is averaged over the samples, so that the size of the cost and its gradient does not depend on the number of samples or on the minibatch size.

3. Multiclass cross entropy

With K classes, such as the ten digits of MNIST, the model instead gives a score z_{ik} for each sample i and class k, and the softmax function turns these scores into probabilities,

$$p_{ik} = \frac{e^{z_{ik}}}{\sum_{l=0}^{K-1} e^{z_{il}}}, \quad (6)$$

which are positive and sum to one for each sample. With one-hot targets, where $y_{ik} = 1$ if sample i belongs to class k and 0 otherwise, the *multiclass cross entropy* becomes

$$C = -\frac{1}{n}\sum_{i=0}^{n-1}\sum_{k=0}^{K-1} y_{ik}\log p_{ik}. \quad (7)$$

Only the term of the correct class is non-zero, so the cost is the average of $-\log$ of the probability given to the correct class. Using $\partial p_{ik}/\partial z_{il} = p_{ik}(\delta_{kl} - p_{il})$ and $\sum_k y_{ik} = 1$, the gradient becomes [6]

$$\frac{\partial C}{\partial z_{ik}} = \frac{1}{n}(p_{ik} - y_{ik}). \quad (8)$$

For $K = 2$ the softmax reduces to the sigmoid, and we recover the binary cross entropy above.

4. Regularisation

Any of these cost functions can be extended with a penalty on the size of the parameters, as in Ridge and Lasso regression [5]. Collecting all weights and biases in a vector $\boldsymbol{\theta}$, the L_2 and L_1 penalties give

$$C_{L_2} = C + \lambda\|\boldsymbol{\theta}\|_2^2, \qquad C_{L_1} = C + \lambda\|\boldsymbol{\theta}\|_1, \quad (9)$$

where $\lambda \geq 0$ sets the strength of the penalty, and the gradients become

$$\nabla C_{L_2} = \nabla C + 2\lambda\boldsymbol{\theta}, \qquad \nabla C_{L_1} = \nabla C + \lambda\,\mathrm{sign}(\boldsymbol{\theta}). \quad (10)$$

The L_2 term pulls each parameter towards zero in proportion to its size, while the L_1 term pulls with a constant force and can set parameters exactly to zero. Since the absolute value is not differentiable at zero, we use $\mathrm{sign}(0) = 0$ there, which is a valid subgradient. Unlike Ridge and Lasso, which leave the intercept unpenalised, we apply the penalty to all weights and biases. It enters only through the gradients, so the MSE and accuracy values we report never include it.

C. Neural networks

1. From linear regression to neural networks

Linear regression methods such as Ordinary Least Squares (OLS), Ridge and Lasso are simple, yet effective methods for fitting data. However, they all share the same shortcoming: the features in the design matrix $X \in \mathbb{R}^{n\times p}$ (where p is the number of features), used to fit the data $\mathbf{y} \in \mathbb{R}^n$, must be defined in advance. For simple problems, this is not an issue, as one can, for example, assume that data can be fitted using a polynomial of a given degree. This, however, does not work for all problems. In particular, due to its poles at $\pm i/5$, high-degree

polynomial fits of the Runge function start to oscillate violently near the ends of the interval, as $x \to \pm 1$. Neural networks get around this problem by learning their own features from the raw input data.

The last layer of a *feed-forward network* (see section II C 2) works much like a linear model. The difference is that the network's features are computed by the earlier layers and learnt from the data together with the final coefficients. The *universal approximation theorem* guarantees that a single hidden layer with a sufficient number of nodes can approximate any continuous function on a closed and bounded domain arbitrarily well [1, 6]. That said, the theorem only guarantees the existence of such a hidden layer, not how many nodes are needed. Neither does it guarantee that gradient descent will find the right weights.

2. The feed-forward pass

Following the notation of the lecture notes, a network with L layers has a weight matrix $W^l \in \mathbb{R}^{n_{l-1} \times n_l}$ and a bias vector $\mathbf{b}^l \in \mathbb{R}^{n_l}$ in each layer $l = 1, \ldots, L$, where n_l is the number of nodes in layer l and $n_0 = p$ [6]. Starting from the input vector $a^0 = x \in \mathbb{R}^p$, each layer computes

$$\mathbf{z}^l = \left(W^l\right)^T \mathbf{a}^{l-1} + \mathbf{b}^l, \qquad \mathbf{a}^l = f^l(\mathbf{z}^l), \tag{11}$$

where f^l is the *activation function* at layer l. In practice, a whole minibatch is propagated at once. With n samples stored in the rows of the input matrix $X \in \mathbb{R}^{n \times p}$, we replace a^0 by $A^0 = X$, so that

$$Z^l = A^{l-1} W^l + \mathbf{b}^l, \qquad A^l = f^l(Z^l), \tag{12}$$

where the bias is added to every row and f^l acts element by element. The sigmoid, ReLU or Leaky ReLU function are used as activation functions (see section II C 3 below) in the hidden layers ($l < L$), while the output layer uses the identity for regression and the softmax function for classification. The final activations A^L are thus the predictions $\tilde{\mathbf{y}}$ or the class probabilities. In the special case of an absence of hidden layers ($L = 1$), a network with a softmax output is exactly multinomial logistic regression.

3. Activation functions

The activation functions make the network non-linear. Without them, the whole network would be reduced to a single linear map. As mentioned in section II C 2, the sigmoid, the ReLU and the Leaky ReLU,

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \tag{13}$$

$$\sigma'(z) = \sigma(z)\left(1 - \sigma(z)\right), \tag{14}$$

$$\mathrm{ReLU}(z) = \max(0, z), \tag{15}$$

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0, \end{cases} \tag{16}$$

$$\mathrm{LReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases} \tag{17}$$

$$\mathrm{LReLU}'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z \leq 0, \end{cases}, \tag{18}$$

with $\alpha = 0.01$, are used as activation functions. The sigmoid is smooth and bounded; its derivative is at most $1/4$ and vanishes as $|z| \to \infty$. In *backpropagation*, the error is sent "backwards" from the output layer to the first layer. At each sigmoid layer, it is multiplied by $\sigma'(z) \leq 1/4$, so after a few layers both the error the gradients of the first layers become very small. This is known as the vanishing-gradient problem (Section 8.9 in [6]). Since the derivative of the ReLU function is 1 for all positive inputs, it does not share this problem with the sigmoid. On the other hand, the ReLU encounters a different issue, namely that a ReLU unit whose input is always negative gets zero gradient and stops learning, a so-called dead unit. The Leaky ReLU circumvents this by keeping a small slope α for negative inputs.

4. Backpropagation

To use gradient descent, we need the derivative of the cost with respect to every weight and bias. Backpropagation computes all of them with one forward pass, followed by one backward pass through the network, by applying the chain rule layer by layer [6, 7]. This is equivalent to reverse-mode automatic differentiation applied to the network.

The error of layer l, $\Delta^l = \partial C / \partial Z^l$, a matrix with the same shape as Z^l, is a key quantity in the process of backpropagation. For a matched pair of output activation and cost function, e.g. a linear output with $C = \mathrm{MSE}/2$, or a softmax output together with the cross entropy, the error of the output layer is

$$\Delta^L = \frac{1}{n}\left(A^L - Y\right), \tag{19}$$

where Y holds the targets, one-hot encoded for classification. For the linear output, $A^L = Z^L$ and Eq. (19) simply becomes the gradient of MSE/2. For the softmax output, Eq. (19) is the gradient of Eq. (7), given element-wise by Eq. (8). We use that $Z^{l+1} = A^l W^{l+1} + \mathbf{b}^{l+1}$ depends on Z^l only through $A^l = f^l(Z^l)$ to move back one layer.

Then by the chain rule, $\partial C/\partial A^l = \Delta^{l+1}(W^{l+1})^T$. Since f^l acts element by element,

$$\Delta^l = \left(\Delta^{l+1}(W^{l+1})^T\right) \odot f'(Z^l), \qquad l = L-1, \ldots, 1, \tag{20}$$

where $\odot$ denotes element-wise multiplication. Finally, Z^l has a linear dependence on W^l and $\mathbf{b}^l$. Hence, the gradients with respect to the weights and biases follow directly,

$$\frac{\partial C}{\partial W^l} = (A^{l-1})^T \Delta^l, \qquad \frac{\partial C}{\partial \mathbf{b}^l} = \sum_i \Delta^l_i, \tag{21}$$

where Δ^l_i denotes the i-th row of Δ^l, i.e. the i-th sample. A penalty adds the terms of Eq. (10). At each step of the backward pass, the same matrix products as in the forward pass are involved. Thus, regardless of how many parameters the network has, the full gradient costs roughly as much as two forward passes. The choice of matched pairs is important. For example, using a sigmoid output and the MSE as a cost function, Eq. (19) gets an extra factor $\sigma'(z)$, and the gradients would be very small exactly when the network is confidently wrong.

5. *Weight initialisation and scaling*

For ReLU and Leaky ReLU, the weights are drawn from a normal distribution with zero mean and variance $2/n_{l-1}$ (He initialisation). For the sigmoid, the variance is $1/n_{l-1}$ (LeCun initialisation) [6]. This way, the variance of the activations stays roughly constant from layer to layer, so that the signal neither dies out nor explodes in deep networks. The biases are set to 0.01, a small positive value, chosen to help the ReLU units start out active. Scaling the input is also important. To avoid having the first layer start out in the flat regions of the sigmoid, we standardise x for the Runge function and divide the MNIST pixel values by 255.

D. Optimisation

We train the network with the gradient methods from project 1 [2]. In particular, plain gradient descent (GD) is used to update all parameters $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \nabla C(\boldsymbol{\theta}_t), \tag{22}$$

where $\nabla C(\boldsymbol{\theta}_t)$ is the gradient of the cost on the full training set and γ is the learning rate. Stochastic gradient descent (SGD) does the same, but achieves this by estimating the gradient from a random minibatch of M samples. At the beginning of every epoch the data are shuffled and split into n/M minibatches, so SGD makes n/M updates per epoch, whereas GD makes only one. The noise in the minibatch gradients can help SGD escape flat regions and poor local minima.

RMSprop and Adam adapt the step size of each parameter individually [6, 8]. RMSprop keeps a running average of the squared gradient,

$$\mathbf{s}_t = \rho \mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2, \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \delta}, \tag{23}$$

where $\mathbf{g}_t$ is the minibatch gradient and all operations are element-wise. There is an inverse relationship between the size of the gradient of a parameter and the size of the step it takes: parameters with consistently large gradients take smaller steps, and those with small gradients take larger ones. Additionally, Adam keeps a running average of the gradient itself, using it as a momentum term,

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \tag{24}$$

$$\mathbf{s}_t = \beta_2 \mathbf{s}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \tag{25}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \gamma \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{s}}_t} + \delta}, \tag{26}$$

with the bias-corrected averages $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$ and $\hat{\mathbf{s}}_t = \mathbf{s}_t/(1-\beta_2^t)$. We use $\rho = 0.99$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\delta = 10^{-8}$, where δ prevents division by zero.

The cost function of a neural network is not convex, unlike that of linear regression. This is an important distinction from project 1. Hence, convergence of the optimisation is dependent on the learning rate, the initialisation and the minibatch size. Furthermore, as there might be multiple minima, these parameters also determine which minimum the optimisation converges to.

E. Data and model selection

For the regression problem we reuse the data and the train/test split of project 1 [2]: $n = 100$ points $x \sim U[-1, 1]$ with $y = f(x) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 0.1^2)$, split into 70 training and 30 test points (seed 2026). We could have generated more data, but we kept the same data set so that we can compare directly with project 1. The input x is standardised and y centred, using the training statistics only. Since the noise has variance $\sigma^2 = 0.01$, no model can be expected to reach a test MSE much below 0.01, and we use this value as a reference throughout.

For classification we use the full MNIST data set [3], with 60 000 training and 10 000 test images of 28×28 grey-scale pixels. Each image is flattened to a vector of 784 features and divided by 255, so that all features lie in $[0, 1]$.

The network has many hyperparameters: the learning rate, the optimiser, the number of epochs, the depth and width, the activation function and the penalty. All of them are chosen on the training data alone. For the Runge function we use five-fold cross-validation (CV). The training set is split into five parts, each model is trained on four of them and evaluated on the fifth, and the CV MSE is the average over the five rounds. The

search itself is run with Scikit-Learn's `GridSearchCV` [4]. For MNIST, where CV on 60 000 images would be expensive, we instead set aside 10 000 of the training images as a validation set, stratified so that each digit has the same share in both sets. In both cases the selected model is refitted on the full training set and evaluated only once on the test set. We vary two hyperparameters at a time around a well-performing setting and show the results as heat maps. Besides the minimum, we also consider the one-standard-error rule, which picks the simplest model whose CV error lies within one standard error of the minimum [5].

The quality of a classifier is measured by its accuracy, the fraction of correctly classified images,

$$\text{Accuracy} = \frac{1}{n}\sum_{i=0}^{n-1} I(\hat{y}_i = y_i), \tag{27}$$

where y_i is the true label, $\hat{y}_i$ the predicted label and I the indicator function, which is 1 when its argument is true and 0 otherwise. The confusion matrix gives a more detailed picture by counting how often each true digit is predicted as each of the ten digits. From it we compute the precision, the recall and the F_1 score of each digit. The precision is the fraction of images predicted as that digit that are correct, the recall is the fraction of images of that digit that are recognised, and the F_1 score is the harmonic mean of the two. Finally, an accuracy p measured on m images has a standard error of about $\sqrt{p(1-p)/m}$.

F. Implementation

All code is written in Python with NumPy and is available in the folder `Project2/Code` of our GitHub repository. The network is implemented in `Project2b.py`, where `create_layers` sets up the weights and biases, `feed_forward` evaluates Eq. (12) and stores Z^l and A^l for every layer, and `backpropagation` computes the gradients of Eqs. (19)–(21). Slightly simplified, the core of the backward pass reads

```
delta = (a[-1] - Y) / n
for l in reversed(range(len(layers))):
    W, b = layers[l]
    dW = a[l].T @ delta + 2*lmbd2*W + lmbd1*np.sign(W)
    db = delta.sum(axis=0) + 2*lmbd2*b + lmbd1*np.sign(b)
    grads[l] = (dW, db)
    if l > 0:
        delta = (delta @ W.T) * f_prime(z[l - 1])
```

The function `train` then loops over epochs and minibatches and updates the parameters with the GD, RMSprop or Adam rules from project 1; SGD is simply GD applied to minibatches.

For the Runge function we wrapped the network in the class `NeuralNetRegressor`. It takes as inputs the number and size of the hidden layers, the activation function, the learning rate γ, the number of epochs, the minibatch size, the optimiser and the penalty λ. Calling `fit` builds and trains the network on the training data, and `predict` evaluates it on new data. Because the class follows Scikit-Learn's estimator interface, Scikit-Learn's cross-validation tools [4] work with it directly, and this is how we searched for the best λ and γ.

For the handwritten digits we use the class `NeuralNetClassifier` in `Project2e.py`, which uses the same functions with a softmax output and the cross entropy cost; with no hidden layers, it reduces to logistic regression. All results and figures are produced in the notebook `Project2.ipynb`, with seed 2026 for the data split and the weight initialisation.

Finally, we verified the backpropagation against automatic differentiation in JAX [9] (section III A), and compared our classifiers with Scikit-Learn's `LogisticRegression` and `MLPClassifier` [4].

G. Use of AI tools

We used Claude for both the writing of the report and code. More information can be seen in appendix A.

III. RESULTS AND DISCUSSION

We start by verifying the implementation, then turn to regression on the Runge function, and end with the classification of MNIST. The data, the scaling and the model selection are described in section II E.

A. Verification of the implementation

Before any analysis, we checked that our backpropagation computes the correct gradient by comparing it with the gradient that `jax.grad` obtains by automatic differentiation of the same cost function [9]. The test covered 54 cases: three hidden activations (sigmoid, ReLU, Leaky ReLU), two cost functions (MSE with a linear output, cross entropy with a softmax output), zero to two hidden layers, and no penalty, an L_2 penalty or an L_1 penalty. Over all weights and biases, the largest difference was 8.9×10^{-16}, which is floating-point round-off. Any problems in the results below must therefore come from the optimisation and not from the gradient.

B. Regression: comparison with OLS

Table I compares sigmoid networks with one and two hidden layers of 50 and 100 nodes, all trained with plain SGD, with OLS of degree 14, the best polynomial from project 1. Two of the networks, (50, 50) and (50, 100), reach a test MSE of 0.011, below the 0.0145 of OLS and close to the noise level. The other three do much worse, with test MSEs between 0.058 and 0.088. In Figure 1 we see that the two best networks follow the narrow peak of

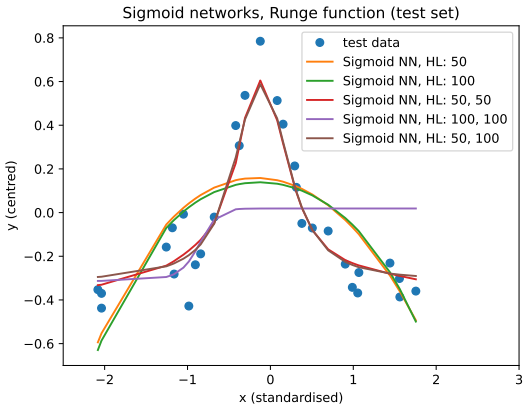


Figure 1. Predictions of the sigmoid networks in Table I on the 30 test points of the Runge function, with standardised x and centred y. All networks are trained with plain SGD (minibatches of 10) for 10 000 epochs with $\gamma = 0.5$. The (50, 50) and (50, 100) networks follow the Runge function most closely.

the Runge function, while the one-layer networks make the peak much too flat, and the (100, 100) network is almost constant over the right half of the interval.

Since all these runs use the same hand-picked learning rate, $\gamma = 0.5$, the spread mainly shows that one learning rate does not suit every architecture. A fair comparison requires tuning the learning rate separately for each of them, which is what we do next.

Table I. Test MSE on the Runge function for OLS of degree 14 (the best result of project 1 on the same data) and for sigmoid networks with a linear output, trained with plain SGD (minibatches of 10) for 10 000 epochs with a fixed learning rate $\gamma = 0.5$. The noise variance is $\sigma^2 = 0.01$.

Model	Test MSE
OLS, degree 14	0.0145
NN (50)	0.0579
NN (100)	0.0607
NN (50, 50)	0.0106
NN (50, 100)	0.0107
NN (100, 100)	0.0883

C. Regression: learning rates and optimisers

Figure 2 shows the CV MSE after 1000 epochs for plain GD, SGD, RMSprop and Adam [8], with learning rates from 10^{-4} to 0.5; GD uses the full training set for each update, the other three minibatches of 10. Many cells lie close to 0.10, which is roughly the variance of y. This is the error of a model that always predicts the mean, so these networks have not learnt anything. This happens for GD and SGD at all learning rates up to 0.01. With small steps and few updates per epoch, the network never leaves the region where the sigmoid units are nearly linear and the output is almost constant.

RMSprop and Adam behave differently. With $\gamma = 10^{-3}$, both reach a CV MSE of about 0.02 for all four architectures, and Adam is the best optimiser for all of them. Their problem is instead large learning rates, where RMSprop becomes unstable, with CV MSEs as large as 199 at $\gamma = 0.5$ in the two-layer networks. SGD, on the other hand, needs a learning rate of 0.1 to 0.5 to be competitive. For the (50, 50) network its best value lies at the edge of our grid, so an even larger learning rate might have helped.

The learning curves in Figure 3 show why GD is so slow. All four optimisers start on a plateau where the network predicts the mean of y, at a cost of about 0.05. SGD, RMSprop and Adam leave it after 50 to 100 epochs and settle below 0.01. GD first overshoots badly, then stays on the plateau until epoch 300 to 500, and has still not converged after 1000 epochs. This is because GD makes only one update per epoch, while the minibatch methods make seven.

With a tuned learning rate, all optimisers except GD give good networks. Table II lists the best learning rate for each architecture and optimiser, together with the test MSE of the model refitted on all training points. Without GD, the test MSE lies between 0.011 and 0.017, and the two-layer networks all reach 0.011, below the 0.0145 of OLS. The CV MSE is consistently higher than the test MSE, partly because each CV model sees only 56 training points, and partly because 30 test points give a noisy estimate. The same noise makes the differences between SGD, RMSprop and Adam on the test set, around 10^{-3}, insignificant. The difference between GD and the other methods, however, is clear.

Table II. Best learning rate γ (lowest five-fold CV MSE after 1000 epochs, from Figure 2) for each sigmoid architecture and optimiser, with the CV MSE and the test MSE of the model refitted on all 70 training points. For comparison, OLS of degree 14 has a test MSE of 0.0145.

Hidden	Optimiser	γ	CV MSE	Test MSE
(50)	GD	0.1	0.0686	0.0595
(50)	SGD	0.1	0.0440	0.0171
(50)	RMSprop	0.001	0.0216	0.0119
(50)	Adam	0.001	0.0187	0.0118
(100)	GD	0.1	0.0388	0.0309
(100)	SGD	0.1	0.0329	0.0170
(100)	RMSprop	0.001	0.0249	0.0126
(100)	Adam	0.001	0.0196	0.0123
(50, 50)	GD	0.5	0.0323	0.0222
(50, 50)	SGD	0.5	0.0230	0.0107
(50, 50)	RMSprop	0.001	0.0214	0.0108
(50, 50)	Adam	0.001	0.0188	0.0110
(100, 100)	GD	0.1	0.0919	0.0927
(100, 100)	SGD	0.1	0.0220	0.0106
(100, 100)	RMSprop	0.001	0.0204	0.0108
(100, 100)	Adam	0.001	0.0186	0.0110

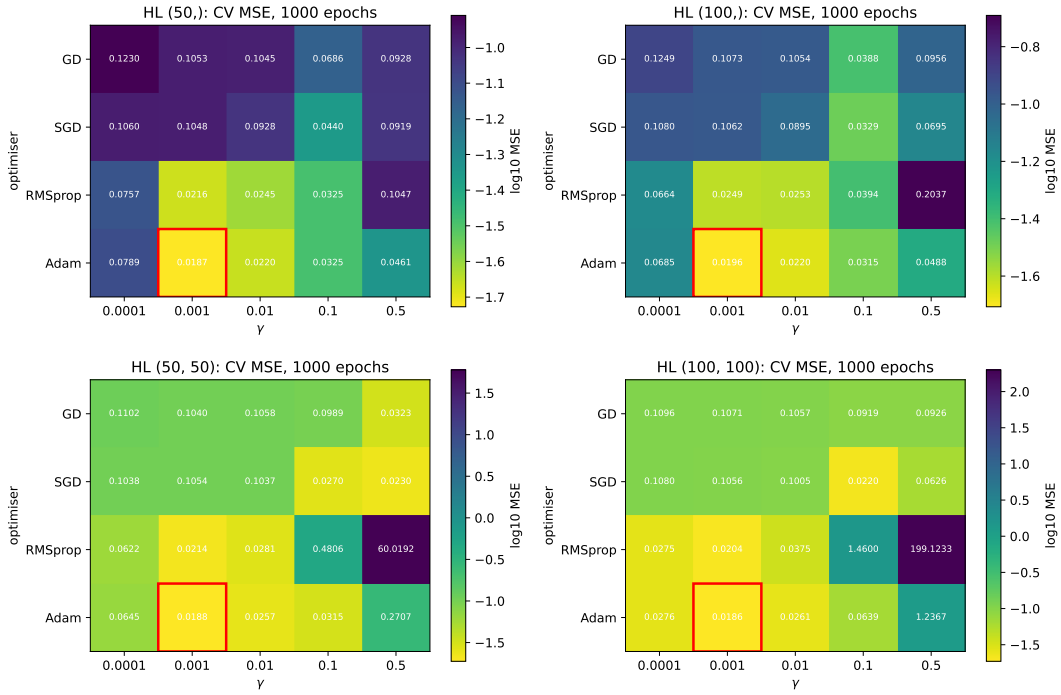


Figure 2. Five-fold cross-validated MSE on the Runge function after 1000 epochs, as a function of optimiser and learning rate γ, for sigmoid networks with one or two hidden layers of 50 or 100 nodes. GD uses the full training set per update; SGD, RMSprop and Adam use minibatches of 10. The colour scale is $\log_{10}$ of the MSE, and the red box marks the lowest value in each panel. Values near 0.10 correspond to a network that predicts the mean of y.

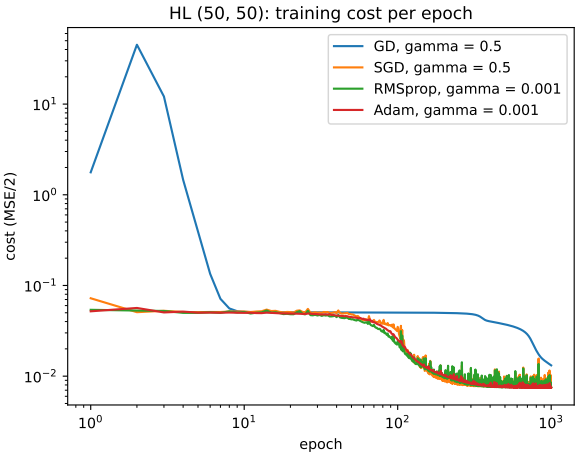


Figure 3. Training cost (MSE/2) per epoch for the (50, 50) sigmoid network on the Runge function, for each optimiser at its best learning rate from Figure 2. A cost of 0.05 corresponds to predicting the mean of y.

D. Regression: activation functions, depth and width

When the networks are trained with plain SGD, the choice of activation function makes a large difference. Figure 4 shows the test MSE for one to five hidden layers after 1000 epochs. With ReLU and Leaky ReLU, the error falls steadily with depth, from about 0.030 with one hidden layer to about 0.011 with five. The two curves are almost identical, which suggests that few ReLU units "die" with the initialisation, so the small negative slope of the Leaky ReLU rarely comes into play.

The sigmoid networks behave very differently. With one hidden layer the test MSE is 0.078, and with two or more it stays at the level of a constant prediction. This is the vanishing-gradient problem explained in section II C 3. Because the derivative of the sigmoid is at most 1/4, the error signal shrinks every time it passes back through a sigmoid layer, and with plain SGD and a fixed learning rate the first layers hardly change. Training one network takes from about 0.5 s with one hidden layer to about 4 s with five.

With Adam, the vanishing gradient problem disappears. Figure 5 shows the CV MSE for one to five hidden layers of 5 to 100 nodes after 500 epochs with Adam, and here all three activations work, with every configuration between 0.020 and 0.051. As seen in section II D, Adam scales each update by a running estimate of the size of the gradient, which compensates for the small gradients of the sigmoid.

In all three panels the best cell is a network with 5 nodes per layer, and the lowest value overall, 0.020, belongs to a single sigmoid layer. The differences between

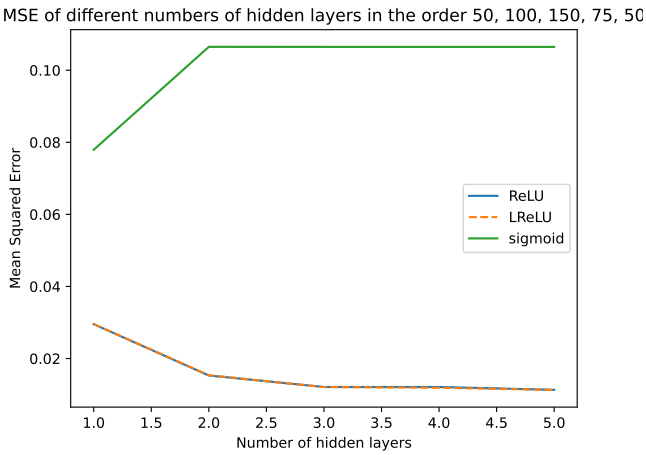


Figure 4. Test MSE on the Runge function as a function of the number of hidden layers, with layer widths 50, 100, 150, 75 and 50 added in that order. All networks are trained with plain SGD (minibatches of 10) for 1000 epochs, with $\gamma = 0.01$ for ReLU and Leaky ReLU and $\gamma = 0.5$ for the sigmoid. The ReLU and Leaky ReLU curves overlap.

the best cells are, however, smaller than the uncertainty of the CV estimate, about 0.005. The main conclusion from the heat maps is therefore that a small network is enough for this one-dimensional problem, and that a single wide layer of 100 nodes does worst.

To look for overfitting, Figure 6 shows the training and test MSE after 3000 epochs with Adam, without early stopping, as the networks grow wider or deeper. We see surprisingly little overfitting. Apart from the narrowest ReLU and Leaky ReLU networks, which underfit, both errors stay between 0.010 and 0.019. The training error never drops well below $\sigma^2 = 0.01$, and with one hidden layer the test error does not grow with the width. The only sign of overfitting is for the sigmoid, where the test error rises from 0.012 with one layer to about 0.018 with four to six, while its training error stays near 0.012.

This is far less overfitting than we saw for the polynomials of project 1 [2], even though the networks have up to about 13 000 parameters and only 70 training points. One possible explanation is that gradient descent, started from small weights, found a smooth function before it began to fit the noise. The test error is sometimes below the training error, which shows how noisy an estimate based on 30 test points is.

E. Regression: L_2 and L_1 penalties

In our first search over the penalties we used 100 epochs, which turned out to be too few. With plain SGD, the lowest CV MSE was 0.092 for L_2 and 0.081 for L_1, not much better than a constant prediction. This agrees with Figure 2, where plain SGD needed many epochs to learn, so we increased the number of epochs to 1000, as in the cross-validation exercises of week 44.

Figure 7 shows the result for the (50, 50) sigmoid network with an L_2 or L_1 penalty λ on all weights and biases. Again, the learning rate is the most important parameter: for $\gamma \leq 0.01$ the network does not learn, $\gamma = 0.7$ is too large, and $\gamma = 0.5$ is best. For both penalties the minimum lies at $\lambda = 0$, with a CV MSE of 0.0229 ± 0.0061. A small penalty ($\lambda = 10^{-5}$) changes nothing, while $\lambda \geq 10^{-3}$ increases the error, for L_1 at large learning rates by several orders of magnitude.

The one-standard-error rule would instead pick $\gamma = 0.1$ and $\lambda = 10^{-5}$. We nevertheless keep the minimum ($\gamma = 0.5$, $\lambda = 0$) for the final models. The rule is meant to favour a more strongly regularised model, but the penalty it selects here is too small to have any effect. The real difference lies in the learning rate, which says nothing about how complex the model is.

This is what we would expect from Figure 6. A penalty reduces the variance at the cost of more bias, and since the network hardly overfits, there is little variance to reduce. Varying the architecture does not change the conclusion (Figure 8): the (50, 50) network without penalty remains the best, and small penalties only help the poorly converged (100) and (100, 100) networks, and then only slightly.

This architecture search should be read with care. It uses the learning rate $\gamma = 0.5$, which was tuned for the (50, 50) network; at this value the one-layer networks perform poorly, and some combinations with large λ diverge (the white cells on the heat map). Tuning the learning rate for each architecture would give a fairer comparison.

Table III compares the penalised networks with Ridge and Lasso regression from project 1 on the same data. Because the optimal λ is zero, the L_2 and L_1 networks are the same model, with a test MSE of 0.0107, against 0.0129 for Ridge and Lasso and 0.0145 for OLS. That the best penalty is zero is also a result in itself. Ridge and Lasso improve on OLS by reducing the variance of the high-degree polynomials, but for the network there is not much variance to remove.

Note that Ridge and Lasso were, as in project 1, selected by their test MSE, whereas the network was selected by cross-validation, so if anything the comparison favours the linear methods. Even so, the differences are small compared with the uncertainty of a 30-point test set, and all four models lie within a factor 1.5 of the noise level.

Table III. Test MSE on the Runge function for the best penalised networks (chosen by five-fold CV, Figure 8) and for OLS, Ridge and Lasso regression with the conventions of project 1 (degree and λ chosen by the test MSE). Since the optimal penalty is $\lambda = 0$, the L_2 and L_1 networks coincide.

Model	Test MSE
NN (50, 50), L_2 or L_1, $\lambda = 0$	0.0107
Ridge, degree 10, $\lambda = 3 \times 10^{-6}$	0.0129
Lasso, degree 13, $\lambda = 10^{-4}$	0.0129
OLS, degree 14	0.0145

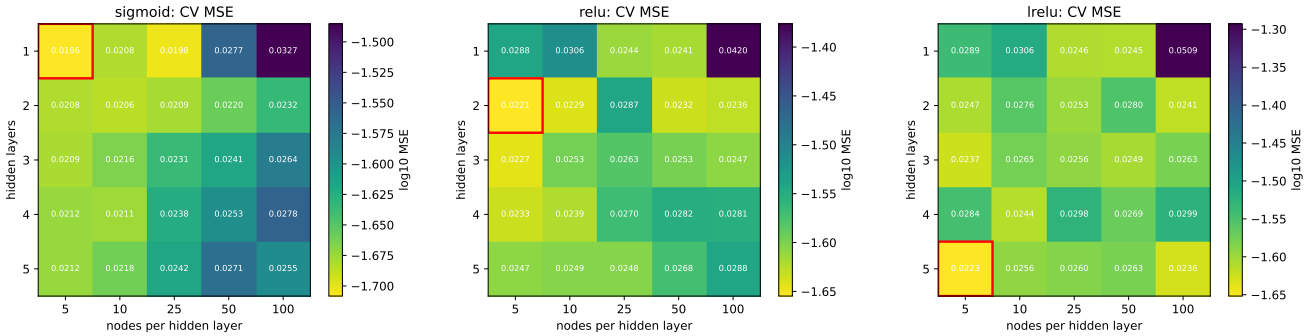


Figure 5. Five-fold cross-validated MSE on the Runge function for 1 to 5 hidden layers of 5 to 100 nodes, for the sigmoid, ReLU and Leaky ReLU activations. All networks are trained with Adam ($\gamma = 0.01$, minibatches of 10) for 500 epochs. The colour scale is $\log_{10}$ of the MSE and differs between the panels; the red box marks the lowest value in each panel.

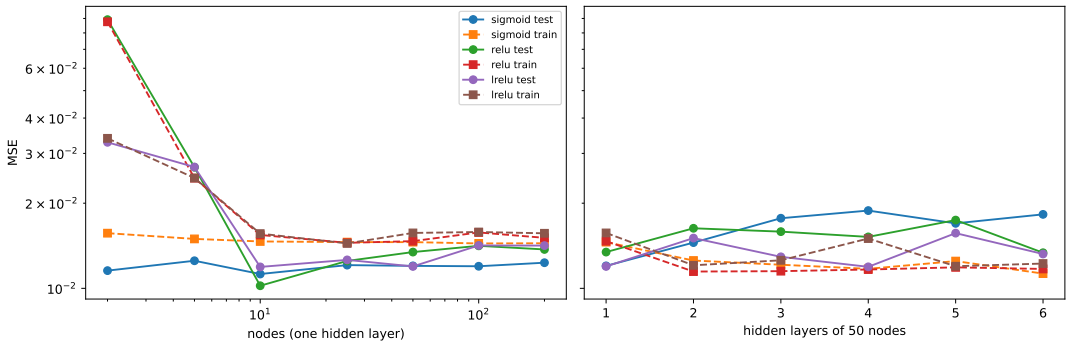


Figure 6. Training (dashed) and test (solid) MSE on the Runge function after 3000 epochs with Adam ($\gamma = 0.01$, minibatches of 10), without early stopping. Left: one hidden layer of 2 to 200 nodes. Right: 1 to 6 hidden layers of 50 nodes. The noise variance is $\sigma^2 = 0.01$.

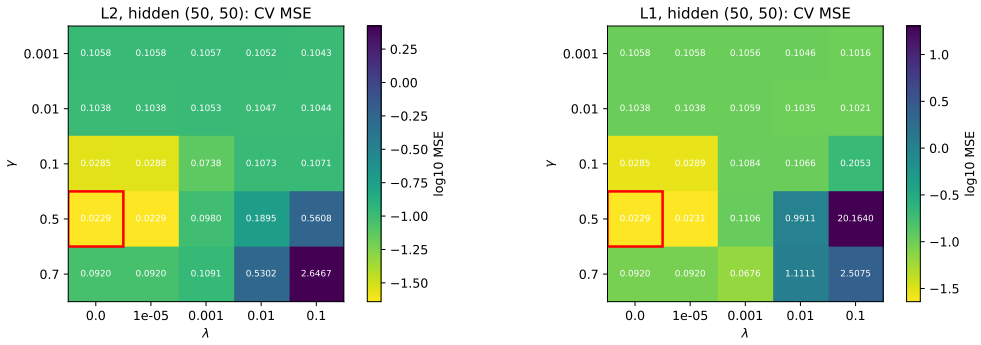


Figure 7. Five-fold cross-validated MSE of a (50, 50) sigmoid network on the Runge function with an L_2 (left) or L_1 (right) penalty λ on all weights and biases, as a function of the learning rate γ and λ. Plain SGD with minibatches of 10, 1000 epochs. The colour scale is $\log_{10}$ of the MSE, and the red box marks the minimum.

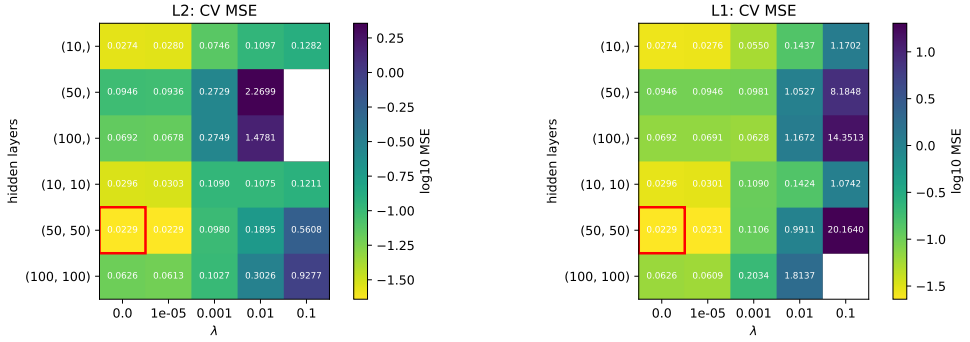


Figure 8. Five-fold cross-validated MSE on the Runge function as a function of the architecture and the penalty λ, for L_2 (left) and L_1 (right), at the learning rate $\gamma = 0.5$ that was best for the (50, 50) network in Figure 7. Sigmoid hidden layers, plain SGD with minibatches of 10, 1000 epochs. White cells: training diverged.

F. Classification of MNIST

Our baseline for MNIST is logistic regression, that is, our network without hidden layers. Trained with plain SGD for 15 epochs, it reaches its best validation accuracy, 0.924, at $\gamma = 0.3$. Figure 9 compares it with a network with one hidden layer of 100 ReLU units, which reaches about 0.977 at the best learning rate whether it is trained with SGD, RMSprop or Adam. One hidden layer therefore improves the accuracy by about five percentage points. As for the Runge function, plain SGD needs a learning rate about a hundred times larger than the adaptive methods.

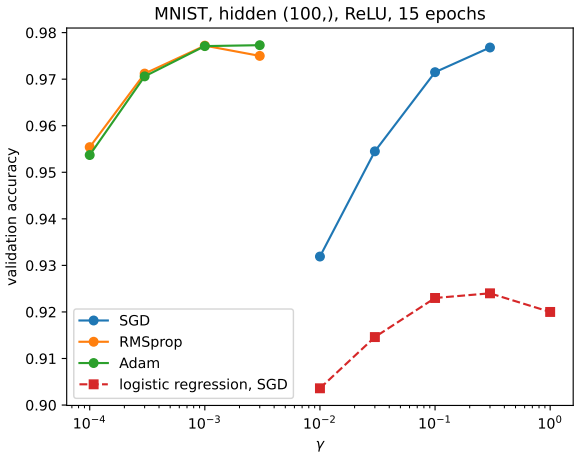


Figure 9. MNIST validation accuracy after 15 epochs (minibatches of 100) as a function of the learning rate, for a network with one hidden layer of 100 ReLU units trained with SGD, RMSprop and Adam, and for logistic regression (no hidden layers) trained with SGD. The validation set is 10 000 images taken from the training set.

The heat maps in Figure 10 confirm that the learning rate matters most. For the (100) ReLU network with Adam, $\gamma = 10^{-4}$ gives 0.954 after 15 epochs, while $\gamma = 10^{-3}$ and 3×10^{-3} both give 0.977; since the best value lies at the edge of the grid, a larger γ would have been worth trying. The L_2 penalty does not help, and $\lambda = 10^{-3}$ lowers the accuracy to 0.963. Just as for the regression problem, the network does not overfit noticeably within 15 epochs, so the penalty only adds bias.

The activation function and the architecture matter less. With the best γ and $\lambda = 0$, all combinations lie between 0.968 and 0.979, and the best of them, a single sigmoid layer of 200 nodes at 0.979, is only just ahead of the best Leaky ReLU network at 0.978. With 10 000 validation images, the standard error of an accuracy around 0.977 is about $\sqrt{0.977 \cdot 0.023/10\,000} \approx 0.0015$, so differences of this size do not single out any activation as better.

Table IV gives the test accuracies of the final models, refitted on all 60 000 training images. Our network with one hidden layer of 200 sigmoid units reaches 0.980, against 0.926 for our logistic regression, so the hidden layer cuts the error rate from 7.4 % to 2.0 %, almost a factor of four.

Both results agree well with Scikit-Learn [4]. Its logistic regression gives 0.9257, against our 0.9256, and its `MLPClassifier` with the same architecture, optimiser, learning rate and number of epochs gives 0.977. The small difference may come from its different weight initialisation; Scikit-Learn also warned that the MLP had not converged after 15 epochs. Our accuracies are in line with the typical values quoted in the project description: about 92 % for logistic regression on raw pixels, and above 99 % for convolutional networks. Convolutional networks make use of the two-dimensional structure of the images, which a fully connected network does not.

Table IV. MNIST test accuracy (10 000 test images, evaluated once) of the final models, refitted on all 60 000 training images. Our network: one hidden layer of 200 sigmoid units, Adam with $\gamma = 3 \times 10^{-3}$, $\lambda = 0$, 15 epochs, minibatches of 100. Logistic regression: SGD with $\gamma = 0.3$, 15 epochs. The Scikit-Learn MLP uses the same settings as our network.

Model	Test accuracy
Own network, (200) sigmoid	0.9800
Scikit-Learn `MLPClassifier`	0.9769
Own logistic regression	0.9256
Scikit-Learn `LogisticRegression`	0.9257
Random guessing	0.10

The confusion matrix in Figure 11 shows where the remaining 200 errors occur. The most common mistakes are 2 predicted as 8 and 9 predicted as 4 (8 cases each), followed by 5 predicted as 3 and 9 predicted as 5 (7 cases each). These digits have similar shapes: a 9 with an open loop resembles a 4, and a 5 and a 3 differ mainly in the upper half. The digit 9 has the lowest recall, 0.967, while a 1 is almost always recognised (recall 0.991). Precision, recall and F_1 score are at least 0.967 for all digits, so the errors are spread fairly evenly. This is information that the accuracy alone does not give.

G. Critical evaluation

According to our results, a linear method is the better choice for the Runge function. The network does reach a lower test MSE than OLS, Ridge and Lasso (0.011 against 0.013 to 0.015), but the difference is within the uncertainty of the test set, and the network is more costly. OLS has a closed-form solution, runs in milliseconds and has a single hyperparameter (the polynomial degree). The network, however, requires much longer training (several minutes) and depends on several hyperparameters: the learning rate, the optimiser, the number of epochs, the depth, the width, the activation and the penalty. Some combinations, such as plain GD or the sigmoid with too small a learning rate, fail completely.

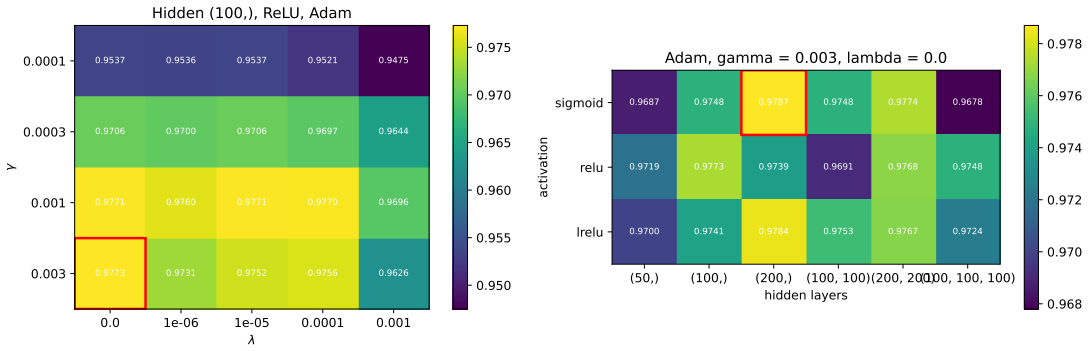


Figure 10. MNIST validation accuracy after 15 epochs with Adam (minibatches of 100). Left: learning rate γ against the L_2 penalty λ for one hidden layer of 100 ReLU units. Right: activation function against architecture, at the best $\gamma = 3 \times 10^{-3}$ and $\lambda = 0$ from the left panel. The red box marks the highest accuracy.

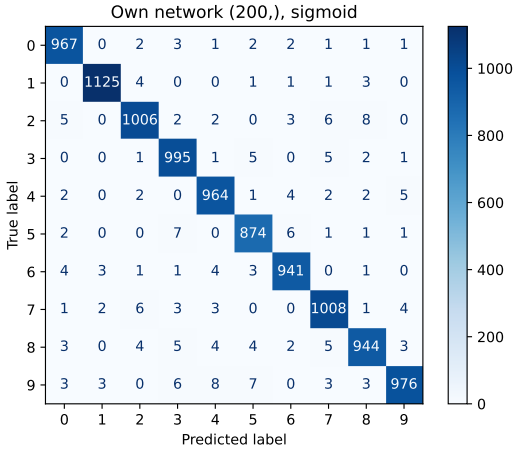


Figure 11. Confusion matrix of our best MNIST model (one hidden layer of 200 sigmoid units, test accuracy 0.980) on the 10 000 test images. Rows give the true digit and columns the predicted digit; the off-diagonal elements are the 200 misclassified images.

The advantage of the network is that it needs no handmade features. It reaches the same accuracy from the raw input x, while OLS needs the powers of x that we chose.

For MNIST the situation is the opposite. Good features are not known in advance, and the improvement from the hidden layer only costs about a second per epoch on 60 000 images, which is well worth it. Among the optimisers, Adam is the most robust, since it gave the best or nearly the best result in every case. RMSprop works equally well as long as the learning rate is small. Plain SGD is competitive once its learning rate is tuned, while plain GD is the least efficient, as it only makes one update per epoch.

Our analysis does have limitations. All results are based on a single seed for both the data split and the weight initialisation, so we do not know how much they would vary between runs. The test set for the Runge function only contains 30 points. Several optimal values lie at the edge of our grids, such as the learning rate for SGD in Figure 2 and for Adam in Figure 10. The architecture search in Figure 8 uses a learning rate tuned for a single architecture.

Some settings that could have been tuned were also kept fixed. The number of epochs was not chosen by early stopping, and the MNIST models were trained for only 15 epochs. The minibatch size was also fixed at 10 for the Runge function and 100 for MNIST, although it affects both the noise in the gradient and the number of updates per epoch. Repeating the runs with several seeds, extending the grids and tuning the number of epochs and the minibatch size would make the conclusions more robust.

IV. CONCLUSION

We have implemented a feed-forward neural network from scratch, with backpropagation, three activation functions, L_2 and L_1 penalties and four optimisers, and verified its gradients against automatic differentiation to within 10^{-15}. The same code handles both regression on the noisy Runge function and classification of the MNIST digits; only the output activation and the cost function change. When they are chosen as matched pairs, a linear output with the MSE and a softmax output with the cross entropy, the error at the output layer is simply the prediction minus the target in both cases.

For the Runge function, our best networks reached a test MSE of about 0.011, close to the noise level and slightly below OLS, Ridge and Lasso from project 1. However, with only 30 test points, we cannot say that the network beats the linear methods, only that it matches them. The networks also overfitted very little, which is probably why neither penalty improved the results.

The optimiser and the learning rate mattered more than the architecture. With small learning rates, Adam and RMSprop trained every architecture well, although RMSprop became unstable at large ones. Plain GD, and

SGD with a small learning rate, often learnt nothing within 1000 epochs. Deep sigmoid networks trained with plain SGD suffered from vanishing gradients, but the problem disappeared when we used ReLU, Leaky ReLU or Adam instead. This shows the difference between what a network can represent and what it can actually learn. The universal approximation theorem guarantees that a network with enough hidden nodes can approximate the Runge function arbitrarily well, but not that gradient descent will find the right weights [6].

For MNIST, a network with one hidden layer of 200 sigmoid nodes reached a test accuracy of 98.0 %, compared with 92.6 % for logistic regression, and both agree well with Scikit-Learn. Most of the remaining errors are between digits that look alike, such as 9 and 4, or 5 and 3.

Our main conclusion is that neural networks are most useful when we do not know in advance which features are good. For a smooth function of one variable, a polynomial fit is faster, simpler and just as accurate. For images, the network gives a large improvement over a linear model, but at the price of many hyperparameters that have to be tuned with care.

Because of limited time and computing power, we could not refine our models as far as we would have liked, and the main limitations are discussed in Section III G. The most important improvements would be to repeat the runs with several seeds, to choose the number of epochs by early stopping, and to tune the minibatch size and the learning rate for each architecture. For MNIST, a convolutional network, which makes use of the two-dimensional structure of the images, is the natural next step towards accuracies above 99 %.

Appendix A: Use of AI/LLM tools

A.1 Tools used. Claude (Claude Opus 5.5, accessed via claude.ai, October–November 2026)

Summary.

A.2 Text writing and editing. The amount of AI used in writing this report can be seen in Table V, with the levels listed below:

- 0: No AI was used writing this
- 1: Language only
- 2: Editorial. We asked AI to rewrite our own sentences.
- 3: Generated. We gave the LLM a prompt and it generated a text. We then revised it to make sure it was correct.

Table V. The level of AI used in our report, per section

Section	Level	Notes
Abstract	2	Asked AI to summarise the project. We rewrote the summary and added what we thought made sense.
Introduction	2	We wrote the draft. AI rewrote it, before we rewrote it again.
Theory and Methods A, B and G	0	written by us
Theory and Methods C-F	2	we wrote the draft, AI rewrote. We then rewrote again
Results and Discussion	3	We asked AI to plug in our results. We edited and verified afterwards
Conclusion	2	Spelling mistakes, structure, rewrite sentences

A.3 Code generation and assistance. The amount of AI used in each script is given in Table VI, with the levels explained below:

- 0: No AI was used.
- 1: We wrote the script and only used AI for debugging
- 2: Most of the script was written by us, but AI helped to write some snippets.
- 3: The skeleton of the code was written by AI, and we wrote on top of that.
- 4: A substantial amount of AI was used to write the code.

Table VI. The level of AI used in our scripts.

File	Level	Description
`utils.py`	4	Taken from previous project[2]. Shared functions: data, scaling, regression, resampling, gradients and optimisers.
`Project1bb.py`	4	Taken from previous project[2]. Needed to compare our data with OLS.
`Project2b.py`	0	No AI was used
`Project2e.py`	0	No AI was used
`Project2.ipynb`	1	AI noticed some small, easy to miss errors in the code, like correct scaling and sizes

All functions from the previous project[2] carry an `LLM-assisted` tag in their docstring.

Many of the functions were also taken from or inspired by previous weekly materials from [10], and we do not know the levels of AI used originally in these functions.

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, 2016).

[2] C. P. Duedahl, O. M. Gården, and I. B. Jónasson, Project 1, FYS-STK3155, University of Oslo (2026).

[3] Y. LeCun, C. Cortes, and C. J. C. Burges, The MNIST database of handwritten digits (2010).

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research **12**, 2825 (2011).

[5] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., Springer Series in Statistics (Springer, New York, 2009).

[6] M. Hjorth-Jensen, Applied data analysis and machine learning, Lecture notes, FYS-STK3155/4155, University of Oslo (2026).

[7] M. A. Nielsen, *Neural Networks and Deep Learning* (Determination Press, 2015).

[8] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, in *3rd International Conference on Learning Representations (ICLR)* (2015).

[9] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, JAX: composable transformations of Python+NumPy programs (2018).

[10] M. Hjorth-Jensen *et al.*, Weekly material for FYS-STK3155 (2026).